

Graph Compression and Multi-View Embedding for Enhanced Community Detection

Meryem Batoul KARIM¹, Menatallahfadoua SLAMA¹, Hiba GUERROUACHE¹, Sabrina CHOUIKRAT¹, Maria DJEBLAHI¹, Meriem DJELLAL¹, Malika BESSEDIK¹, Fatima Benbouzid-Si Tayeb¹

¹*Laboratoire des Methodes de Conception de Systemes (LMCS),*

²*Ecole Nationale Supérieure d'informatique (ESI), BP 68M - 16270 Oued Smar, Algiers, Algeria*
{*km.karim, km.slama, kh.guerrouache, ks.chouikrat, km.djeblahi, jm.djellal, m.bessedik, f.sitayeb*}@esi.dz

Keywords: Community detection, graph embeddings, clustering, label propagation, subgraph extraction, Machine learning .

Abstract: Community detection in networks is a critical task for understanding the underlying structure and functional organization of complex systems. Traditional community detection algorithms have typically relied on topological features and modularity optimization. Over time, the field has significantly evolved with the incorporation of machine learning techniques, which have enhanced the ability to uncover more intricate community structures. Recently, a major focus has shifted towards leveraging graph embeddings to capture geometric representations of networks, providing a more nuanced and comprehensive view of community structures. In this paper, we introduce a novel community detection algorithm, MVEGC (Multi-View Embedding Graph Compression) that leverages multiple similarity matrices to create enhanced node embeddings, capturing various aspects of node relationships. MVEGC simplifies the network, thereby facilitating more accurate community detection. Experimental results on benchmark datasets demonstrate that MVEGC outperforms existing methods in terms of modularity and NMI (Normalized Mutual Information), underscoring its effectiveness in accurately capturing the true community structures within networks.

1 INTRODUCTION

Networks are a powerful tool for representing relational data in the real world. In this context, vertices represent entities, while edges describe the relationships between them. This approach facilitates the analysis of complex systems where connections and interactions are crucial. As the volume of information grows, network data is becoming increasingly prevalent. Social networks (SNs) are a prime example, defined by Wasserman and Faust (1994) as "a set of actors (people, organizations, machines, etc.) and the relationships (interaction ties) among them." (Mazouzi et al., 2019). Social networks, including Online Social Networks (OSNs) like Facebook and LinkedIn, have evolved into intricate structures. They generate vast datasets capturing user interactions and relationships, creating a complex digital landscape.

Social Network Analysis (SNA) has emerged as a critical field dedicated to studying these structures, aiming to understand the underlying communities and connections that bind users within these networks.

A core challenge in SNA is community detection, which involves identifying and analyzing natural groupings, or communities, within complex networks. These communities consist of cohesive groups of nodes (individuals or entities) interconnected by strong ties. Community detection algorithms, designed to partition nodes into such groups based on patterns of connectivity or similarity, are crucial for revealing the underlying structure and organization of the network. By uncovering communities, these algorithms facilitate a deeper understanding of network dynamics, the identification of influential nodes, and the detection of functional modules in various domains such as social networks, biological networks, and information networks.

Community detection is inherently complex due to the sheer size and inherent noise present in large-scale social networks, making it difficult to delineate clear community boundaries. Traditionally, community detection algorithms have relied on modularity optimization and other topological features, with the Louvain algorithm being one of the most tra-

ditional and widely used methods (Blondel et al., 2008a). Modularity being a measure of the strength of division of a network into communities; it quantifies the density of links inside communities compared to links between communities. While these methods have been valuable, they often struggle with limitations such as difficulty in handling overlapping communities or communities with weak connections. Furthermore, the Community Detection Problem is NP-hard, meaning that finding the optimal partition for large networks is computationally challenging. This difficulty arises from the lack of a concrete definition of a community (Fortunato, 2010), as there is no absolute way to determine definitively whether a group of nodes belongs to the same community. Instead, various metrics help assess node similarity and identify potential community centers.

The integration of Machine Learning (ML) techniques has significantly transformed the field of community detection. This has led to substantial improvements in both performance and efficiency. These advanced methods effectively address the challenges faced by traditional approaches, particularly when dealing with large, noisy, or weakly connected networks. ML-driven techniques such as deep learning and graph embeddings offer more robust tools. These tools allow researchers to uncover intricate patterns within complex networks, ultimately leading to a deeper and more accurate understanding of their underlying structures.

To build on these improvements and further refine community detection, we can explore a range of existing algorithms in the literature. The study of community detection algorithms has evolved significantly, with a variety of methods emerging over time. Initially, modularity-based algorithms, such as Louvain (Blondel et al., 2008b), as we have already mentioned in the previous paragraphs and its enhanced version (Zhang et al., 2021), set the stage by optimizing modularity to efficiently partition large networks. Newman's (Newman and Girvan, 2004) pioneering work on these algorithms provided the groundwork for many subsequent advancements. Following this, density-based algorithms like DBSCAN (Zhao et al., 2018) and OPTICS (Subramani et al., 2011) emerged, offering robust methods to identify clusters based on density while effectively handling noise. These algorithms transitioned the focus towards spatial characteristics within networks. The next progression in the field saw the introduction of node similarity-based algorithms, such as those using affinity functions (Idocin et al., 2022) and SVMs (Sui et al., 2016). These methods leverage node attributes and similarities to detect communities, em-

phasizing relational data over structural properties alone. Centroid-based algorithms were subsequently developed, leveraging measures of centrality to discover influential nodes and their corresponding communities. Notable examples include PCMeans (Louaf and Titouna, 2023) and k-means clustering (He et al., 2021), which utilize centroids to identify key nodes and their clusters.

In parallel, Flow-based algorithms like the Label Propagation Algorithm (LPA) (Raghavan et al., 2007) gained popularity for their simplicity and linear time complexity, making them suitable for social network analysis. However, challenges such as randomness and instability during the label propagation process have impacted their accuracy and stability. To address these issues, hybrid algorithms were developed, combining multiple approaches for more robust community detection, notable works in this section include the Random Walk-based Seed Expansion (RWSE) (Su et al., 2017), which integrates different methodologies for improved performance.

Recently, graph embedding-based algorithms have gained traction, with methods such as DEEPWALK (Chen et al., 2020), NODE2VEC (Grover and Leskovec, 2016a), and GEMSEC (Rozemberczki et al., 2019) utilizing advanced embedding techniques to capture network features in low-dimensional spaces. This latest evolution reflects the increasing sophistication and capability of community detection algorithms in addressing complex network structures.

Several recent studies have advanced community detection in complex networks using machine learning techniques. (Xie et al., 2019) introduced CDDTA, which uses a deep transitive autoencoder and unsupervised transfer learning for effective vertex similarity capture, though it faces challenges in model complexity and computational demands for large networks. (Xu et al., 2020) proposed a method combining stacked autoencoders and ensemble clustering for improved accuracy, but with increased complexity. (Fei et al., 2023) developed CSEA, using a variational autoencoder and K-truss algorithm, excelling in dense, medium-sized networks but requiring further investigation in sparse or large networks. (Leng et al., 2024) presented a semi-supervised learning approach with label propagation, achieving high accuracy with minimal labeled data, though its effectiveness depends on the initial node selection. (Wang et al., 2024) introduced a self-supervised graph autoencoder to reduce redundancy and improve embeddings, benefiting from reduced labeled data but facing computational challenges for large networks.

To overcome the limitations of traditional meth-

ods and unlock a deeper understanding of complex networks, we present a novel **Multi-View Embedded Graph Compression** (MVEGC) framework. MVEGC leverages the power of multi-view embeddings, capturing diverse perspectives on node similarities within the network. This comprehensive view goes beyond traditional approaches, providing a richer understanding of the relationships between nodes. Furthermore, MVEGC incorporates an efficient graph compression technique to tackle the challenges of scale and complexity inherent in large-scale networks. This allows MVEGC to handle massive datasets while maintaining focus on the most relevant structural elements for community detection. By combining the strengths of multi-view embeddings and graph compression, MVEGC offers a robust framework for uncovering intricate community structures within complex networks.

The rest of this paper is organized as follows: In Section II, we define the problem of community detection and outline the challenges addressed by our work. In Section III, we describe our proposed multi-view embeddings with graph compression framework and its underlying algorithms. Experimental results and a comparative analysis with various machine learning methods are detailed in Section IV. Finally, Section V provides the conclusion, summarizing our findings and suggesting avenues for future research.

2 PROBLEM DEFINITION

The Community Detection Problem in networks is a fundamental problem in the field of graph theory and network science. It involves identifying groups of vertices (communities) in a graph that are more densely connected to each other than to the rest of the network. This problem is crucial for understanding the structural and functional properties of complex networks.

Formally, let $G = (V, E)$ denote an undirected graph where:

- V is the set of n vertices (or nodes), $|V| = n$.
- E is the set of m edges, $|E| = m$.
- Each edge $(u, v) \in E$ represents a connection between vertices u and v .

The goal of the Community Detection Problem is to partition the set of vertices V into k disjoint communities $C_1, C_2, \dots, C_k$ such that the vertices within each community are more densely connected to each other than to vertices in other communities. The objective is to find the optimal partitioning of V that

maximizes the intra-community edges while minimizing the inter-community edges.

3 MULTI-VIEW EMBEDDING AND GRAPH COMPRESSION: MVEGC

The overall framework of our proposed graph compression method is illustrated in Fig1, comprising three key components. Specifically, our method utilizes multi-view embeddings generated from different similarity matrices as inputs to an auto-encoder to obtain node embeddings. This approach employs multiple similarity matrices to capture different features of nodes within a graph, enhancing the representational quality of the embeddings and providing a more comprehensive view of the graph's structure. Simultaneously, a graph compression algorithm is applied to extract the most significant nodes, focusing on the structural aspects of the graph. This technique seeks to reduce noise and redundancy, potentially leading to improved clustering results. Subsequently, using the embeddings derived from the initial phase and focusing only on the nodes of the compressed subgraph, an initial clustering of these nodes is performed using the spectral clustering method. Finally, we generalize these clustering results to the remaining nodes of the original graph in a semi-supervised manner via the Modified label propagation method. This approach builds on initial clustering results derived from core nodes and aims to generalize these clusters to the entire graph. The details of these components involved in the model are described below .

Table 1: Notations and explanations

Symbols	Description
G	The original attributed network.
G'	The network subgraph
N	The set of all nodes.
N'	The set of nodes in the subgraph.
d	The size of embeddings vector.
d'	The size all similarity representations vector for a node i.
X	The set of embeddings for the initial graph $N \times d$
X'	The set of embeddings for the subgraph $N' \times d$
A	The normalized Adjacency Matrix
M	The Adamic adar matrix
J	Jaccard matrix

3.1 Multiview Embeddings

Multiview embedding is a sophisticated technique that leverages multiple similarity matrices as inputs to an autoencoder, generating enriched embeddings for nodes in a graph. This method exploits the in-

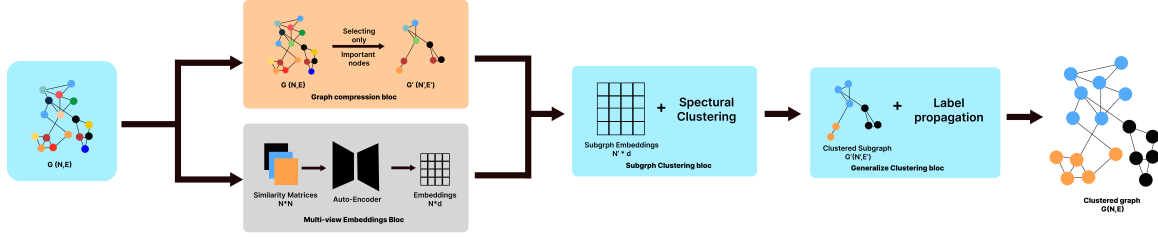


Figure 1: Overview of the proposed framework, consisting of four different modules : (a) Multiview embedding , (b) graph compression , (c) subgraph clustering , (d) clustering generalization

herent diversity of information captured by different similarity matrices, each reflecting unique aspects of the graph’s structure and attributes. These similarity matrices are stacked and fed into the autoencoder, forming the entry point of the model. By integrating these diverse perspectives, the autoencoder produces embeddings that encapsulate a comprehensive understanding of the graph’s topology and node relationships. The rationale behind using multiple similarity matrices is based on the observation that each matrix can highlight different characteristics and patterns within the graph. For instance, one similarity matrix might emphasize structural equivalence between nodes, while another might focus on attribute-based similarities. By feeding these varied similarity measures into the autoencoder, the multiview embedding approach combines these different dimensions of information, leading to more robust and informative node embeddings.

The choice of similarity matrices is vital in multiview embeddings as they provide diverse perspectives on the graph’s structure. These matrices serve as inputs to the autoencoder, enriching the node embeddings and improving community detection. Below, we describe the different similarity matrices used in our method and their unique contributions to the embedding process.

- **Normalized Adjacency Matrix:**

It Captures local graph information by focusing on immediate connections (Shigehalli and Shettar, 2011). Defined as:

$$\hat{A} = D^{-\frac{1}{2}} A D^{-\frac{1}{2}} \quad (1)$$

A is the adjacency matrix, and D is the degree matrix with D_{ii} representing the degree of node i . The normalization scales A by the inverse square root of D to balance node contributions.

- **Jaccard Similarity Matrix:**

Measures similarity between pairs of nodes based on shared neighbors (Hancock, 2004). Each entry

$\text{Jaccard}(i, j)$ represents the Jaccard similarity coefficient between nodes i and j . The coefficient is defined as:

$$\text{Jaccard}(i, j) = \frac{|N(i) \cap N(j)|}{|N(i) \cup N(j)|} \quad (2)$$

Where $N(i)$ and $N(j)$ are the sets of neighbors of nodes i and j , respectively.

- **Adamic-Adar similarity Matrix:**

The Adamic-Adar similarity matrix (Adamic and Adar, 2003) measures the similarity between pairs of nodes based on their shared neighbors, adjusted by the logarithm of the degree of the neighbors. This measure helps identify nodes that share common neighbors, with a preference for neighbors with lower degrees. It highlights important connections within communities, especially for nodes with fewer but more significant connections. Each entry $\text{AdamicAdar}(i, j)$ is defined as:

$$\text{AdamicAdar}(i, j) = \sum_{k \in N(i) \cap N(j)} \frac{1}{\log |N(k)|} \quad (3)$$

Where $N(i)$ and $N(j)$ are the sets of neighbors of nodes i and j , respectively, and $|N(k)|$ is the degree of the common neighbor k .

3.1.1 Model Training

During training, the autoencoder combines information from each similarity matrix by stacking them to form a single input vector x of dimension d' . Mathematically, this is represented as $x = [x^{(1)}, x^{(2)}, \dots, x^{(m)}]$, where m is the number of matrices and d' is the total dimensionality of the input vector. The autoencoder processes this input vector through its encoder and decoder components to produce a lower-dimensional latent space representation X of dimension d , where $d' \gg d$. Throughout training, the autoencoder minimizes the reconstruction error between the input and output vectors by optimizing a loss function : Mean Squared Error. This en-

sures that the learned representations effectively capture the information from the stacked similarity matrices, facilitating the main task of community detection. The output of the autoencoder consists of embeddings, vectors of dimension d , representing the nodes in the graph.

3.2 Graph Compression

In this section, we propose a method to extract a subgraph from the entire graph that provides a clearer view of communities, inspired by the work of Leng et al. (2013) (Leng et al., 2013). The algorithm is divided into two stages. In the first stage, we identify core nodes from the network. While node degree is significant in network analysis, using it alone does not suffice to determine node importance. Instead, we use node density as a more comprehensive measure of importance. The method focuses on selecting nodes from a subset termed the core nodes set, rather than from the entire network. Initially, nodes with lower density, approximately 25% of the total, are eliminated, and the remaining nodes are designated as core nodes. The details of selecting core nodes are shown in Algorithm 1.

- **Definition 1: Density of Node i**

Given a complex network G , the density of a node i is defined as follows,

$$\text{density}(i) = \frac{\sum_{j \in N(i)} n_{ij}}{2 \cdot k_i} \quad (4)$$

where k_i is the degree of node i , $N(i)$ is the set of neighbors of node i , and n_{ij} is defined as,

$$n_{ij} = |N(i) \cap N(j)|$$

Let $\text{density}(G)$ denote the set of all densities of nodes in the complex network G . (Leng et al., 2013)

- **Definition 2: 25th Percentile of a Set S .** Given a set S , the 25th percentile of S is the value such that 25 percent of the elements in S have values less than or equal to it, and 75 percent of the elements in S have values larger than it. (Leng et al., 2013)
- **Definition 3: Core Node.** Given a complex network G , a node i in G is a core node if and only if $\text{density}(i)$ is larger than or equal to the 25th percentile of $\text{density}(G)$. (Leng et al., 2013)
- **Definition 4: Similarity Between Nodes i and j .** Given a complex network G , i and j are two nodes in G . The similarity between i and j is defined as,

$$\text{sim}(i, j) = \frac{n_{ij}}{k_i + k_j} \quad (5)$$

where the meaning of n_{ij} is the same as in Definition 1, and k_i and k_j are the degrees of node i and node j , respectively. (Leng et al., 2013)

In the second stage, we select k nodes from these core nodes. In this paper, our goal is to select as few nodes as possible to achieve optimal detection results. This selection is based on several considerations. First, labeling nodes in a complex network is challenging. Core nodes, being more important and better representatives of communities, are easier to label compared to nodes at the boundaries of communities. The subgraph generated by core nodes has a simpler topology than the entire graph, which may contain noise nodes. By focusing on core nodes, we reduce complexity and improve the clarity of community structures.

To achieve this, we leverage the weighted shortest path method, adapting it to our unweighted graph by assigning weights based on the dissimilarities between nodes. The core idea is to iteratively select nodes that are most dissimilar from the already selected nodes. To do this, we calculate the shortest path distance of each node to the selected nodes and choose the node with the maximum distance to be added to the core nodes.

First, we select two nodes (u and v) with the maximum shortest path value, and identify the node u' with the highest degree within the set $N(u) \cup \{u\}$ as the first selected node. Similarly, we identify the node v' with the highest degree within the set $N(v) \cup \{v\}$ as the second selected node. Both u' and v' are added to the SelectedNodes set. Next, we select the node u from the CoreNodes that is most dissimilar to the selected nodes. This process is repeated until the size of SelectedNodes reaches k . The value of k is determined using the elbow method, ensuring an optimal number of selected nodes.

Finally, the k selected core nodes are considered the final result of our algorithm.

3.3 Clustering Subgraph

After obtaining the subgraph, the next step is to perform clustering. Instead of using a traditional similarity matrix, we employ spectral clustering with embeddings derived from the entire graph. Specifically, we extract the embedding vectors corresponding only to nodes present in the subgraph.

Spectral clustering is a technique that leverages the eigenvalues and eigenvectors of a similarity graph to partition data points into clusters. By applying spectral clustering with embeddings, our approach integrates comprehensive information from the entire

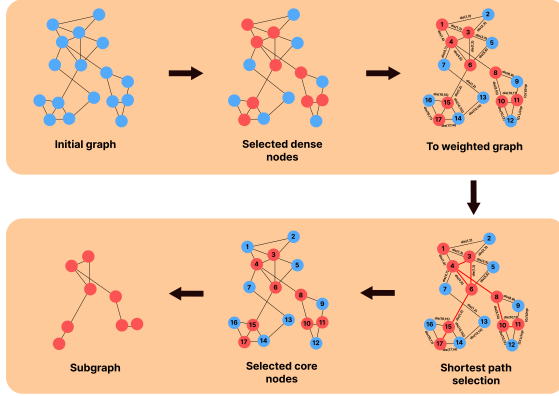


Figure 2: Graph compression method iterations on a simple graph

Data: Graph G , Number of nodes to select k
Result: Set of k selected core nodes
Stage 1: Identify Core Nodes ;
 Identify core nodes from the network based on the density;
 Let $CoreNodes$ be the set of identified core nodes;
Stage 2: Select k Nodes from Core Nodes ;
 Initialize $SelectedNodes$ as an empty set;
Step 2.1: Initialize with Most Dissimilar Nodes ;
 Select nodes u and v with the maximum shortest path value;
 Identify node u' with the highest degree in $N(u) \cup \{u\}$ and add u' to $SelectedNodes$;
 Identify node v' with the highest degree in $N(v) \cup \{v\}$ and add v' to $SelectedNodes$;
Step 2.2: Iterative Selection Based on Dissimilarity ;
while size of $SelectedNodes$ is less than k **do**
 Select node u from $CoreNodes$ that is most dissimilar to nodes in $SelectedNodes$;
 Identify node u' with the highest degree in $N(u) \cup \{u\}$;
 Add u' to $SelectedNodes$;
end
return the set of k selected core nodes in $SelectedNodes$;
 Algorithm 1: Core Node Selection Algorithm

graph. This method enhances clustering accuracy by leveraging the rich relationships and structural insights embedded in the original graph .(von Luxburg, 2007).

3.4 Generalize Clustering

The Modified Label Propagation Algorithm is tailored to identify communities within a given graph and operates in three main stages: initialization, expansion, and labeling.

In the initialization stage, the goal is to get the core nodes with their labels from the previous stage and consider these points as starting points .

During the expansion stage, the primary goal is to expand these initial communities by classifying the direct neighbors of the core nodes. This classification is based on calculating the similarity between the core nodes and their neighbors. If the similarity exceeds a predefined threshold, the neighbor is incorporated into the same community as the core node. If the similarity is below the threshold, the neighbor remains unclassified. The similarity used in this context will be specified below. This step iteratively includes nodes into communities, effectively growing the communities around the core nodes based on their local connectivity and similarity.

In the final labeling stage, any remaining unlabeled nodes are assigned to the community where they exhibit the highest average similarity to nodes already labeled within that community. This ensures that nodes are grouped into communities where their mutual similarity is maximized .The details of the algorithm are shown in algorithm 2 .

- **Cosine Similarity:** Cosine similarity measures the cosine of the angle between two vectors in an inner product space, and is defined as follows:

$$\text{cosine_similarity}(u, v) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} \quad (6)$$

where $\mathbf{u}$ and $\mathbf{v}$ are the embedding vectors of nodes u and v , respectively. Here, $\mathbf{u} \cdot \mathbf{v}$ denotes the dot product of $\mathbf{u}$ and $\mathbf{v}$, and $\|\mathbf{u}\|$ and $\|\mathbf{v}\|$ are the magnitudes of the vectors $\mathbf{u}$ and $\mathbf{v}$ (Manning et al., 2008).

- **Similarity Between a Node and a Group of Nodes:** To measure the similarity between a node u and a group of nodes S , we compute the average cosine similarity between u and each node in S :

$$\text{group_similarity}(u, S) = \frac{1}{|S|} \sum_{v \in S} \text{cosine_similarity}(u, v) \quad (7)$$

where S is the set of nodes representing the community, and $\text{cosine_similarity}(u, v)$ is as defined above (Manning et al., 2008).

4 COMPLEXITY ANALYSIS

Our method has several stages, each with its own computational complexity that we will be detailing below :

1. Creating Similarity Matrices:

Data: Graph G , Initial core nodes with labels from the previous stage

Result: Communities identified in the graph

Initialization: ;

Get core nodes with their labels from the previous stage and treat them as starting points;

Sort core nodes based on the density;

Expansion: ;

while there are unlabeled nodes **do**

for each core node **do**

 Calculate similarity between the core node and its neighbors using cosine similarity;

if similarity exceeds a predefined threshold **then**

 Include the neighbor in the same community as the core node;

else

 Leave the neighbor unclassified;

end

end

end

Labeling: ;

Assign any remaining unlabeled nodes to the community where they have the highest average similarity to labeled nodes;

Algorithm 2: Modified Label Propagation Algorithm

- The creation of similarity matrices involves pairwise comparisons of nodes. therefore the complexity for this step is estimated at $O(n^2)$.

2. Combining Similarity Matrices:

- When combining k similarity matrices, each matrix of size $n \times n$, the operations required have a complexity of $O(n^2 \cdot k)$.

3. Autoencoder Embeddings:

- The autoencoder processes embeddings over multiple epochs. For each epoch, the complexity involves processing n^2 elements plus additional operations proportional to the embedding dimensions, resulting in $O(n^2 + n \cdot d)$. And so the complexity of this step is $O(n^2)$ overall.

4. Subgraph Complexity:

- Extracting a subgraph from the entire graph can be complex, especially when evaluating node connections. This operation's complexity is approximated at $O(n^3)$ due to the use of the shortest weighted path technique used in the algorithm.

5. Spectral Clustering and Label Propagation:

- Spectral clustering involves eigen decomposition of matrices, typically an $O(n^3)$ operation in the worst case. Label propagation, which classifies nodes based on their neighbors, has

a complexity of $O(n^2 \cdot d)$. Combined, the complexity remains $O(n^3)$ overall.

6. Overall Complexity:

- Considering the complexities of all stages, the overall complexity of the solution is dominated by the $O(n^3)$ terms from creating similarity matrices, subgraph extraction, and spectral clustering. Hence, the overall complexity of our method is $O(n^3)$.

5 EXPERIMENTAL STUDY

In this section, we present the experimental results of the MVEGC (Multi-View Embedding Graph Compression) method across various benchmark datasets. We will start by presenting the metrics used to evaluate our results, followed by an overview of the different algorithms we compare our solution with. Next, we will describe the various datasets used for testing. After that, we conduct an empirical study, varying different parameters of our algorithm to identify the optimal values. We also examine the effect of each step or component of the proposed framework. Finally, we provide a comparative study of the results obtained by our method against other methods.

We compared our approach using key clustering metrics: Normalized Mutual Information (NMI) (Danon et al., 2005) and modularity (Clauset et al., 2004) scores, where higher values indicate better performance. This comprehensive analysis demonstrates the efficacy and robustness of MVEGC in different contexts

5.1 Evaluation Metrics

For the community information C'_p and real community information C_q , when the real community labels are known, set $p = q$.

• Normalized Mutual Information (NMI)

Normalized Mutual Information (NMI) (Danon et al., 2005) is a widely used similarity measure, which represents the similarity between real community division and community division obtained by community detection algorithm. The equation is as follows where $p(C_p, C_p')$ is joint probability between clusters and $p(C), p(C_p)$ are marginal probability of clusters :

$$I(C_p, C_p') = \sum_{c_p \in C_p} \sum_{c_p' \in C_p'} p(c_p, c_p') \log \left(\frac{p(c_p, c_p')}{p(c_p)p(c_p')} \right) \quad (8)$$

- **Modularity (Q)**

Modularity (Clauset et al., 2004) is a metric used in network analysis to evaluate the strength and quality of the division of a network into communities. It measures how well the detected communities match the network’s actual structure compared to what would be expected in a random network. Higher modularity indicates a more pronounced community structure. It is defined as :

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j) \quad (9)$$

where A_{ij} is the adjacency matrix entry for nodes i and j , k_i is the degree of node i , $\delta(c_i, c_j)$ is the Kronecker delta function indicating whether nodes i and j are in the same community, and m is the total number of edges.

These metrics are essential for evaluating the performance of community detection algorithms, providing insights into how well the detected communities align with the ground truth community structure when available.

5.2 Used Methods for Comparison

We compared MVEGC with the following methods:

- (Newman and Girvan, 2004) : A hierarchical clustering algorithm on the basis of greedy strategy.
- (Amini and Levina, 2013) : A classical clustering algorithm built on spectral graph theory, which constructs feature vectors by calculating the Laplacian matrix.
- Edmot (Li et al., 2019): A new advanced Motif-aware edge enhancement method for community detection.
- DSACD (Fei et al., 2020): A dimension reduction and feature extraction network structure that uses a similarity matrix and a deep sparse autoencoder, where a low-dimensional representation is obtained and clustered using the K-means algorithm.
- Deepwalk (Perozzi et al., 2014): A network embedding method based on random walks, where a low-dimensional representation is obtained and clustered using the K-means algorithm.
- Node2vec (Grover and Leskovec, 2016b): Based upon the Deepwalk algorithm, node2vec combines both depth-first and breadth-first walks to obtain a low-dimensional representation and clustered using the K-means algorithm.

- NMF (Gillis and Glineur, 2012) : A Nonlinear dimension reduction that is achieved by factorizing the original matrix and clustered using the K-means algorithm.

5.3 Benchmark Datasets

In this section, we present the different datasets used to test the performance of the newly proposed approach. All the datasets leverage typical features of social networks. We have conducted tests on synthetic datasets, and real datasets which are described and presented in Table 2.

Table 2: Statistics of benchmark datasets used for evaluation. n denotes number of nodes, m denotes number of edges, q denotes number of communities.

Dataset	n	m	q
Karate	34	78	2
Polbooks	105	441	3
Football	180	788	12
Dolphins	62	159	2
Email	1005	25571	42

we also tested on The Lancichinetti-Fortunato-Radicchi (LFR) benchmark network which is a synthetic dataset designed to emulate the complex features of real-world networks, surpassing the Girvan-Newman (GN) benchmark, which lacks heterogeneity in vertex degrees and community sizes (Lancichinetti et al., 2008). The LFR benchmark (Li et al., 2008) simulates networks with power-law distributions of node degrees and community sizes, providing a realistic and challenging environment for testing community detection algorithms.

In our experiments, the LFR benchmark was configured with an average degree of 20 and a mixing parameter (μ) ranging from 0.15 to 0.6, to reflect varying levels of community mixing and structural complexity. Notably, when μ reached 0.4 or higher, the community structure became highly complex and less distinct, making it significantly more challenging and nearly impossible to detect communities accurately.

5.4 Hyperparameter Settings

The performance of our method depended on two critical hyperparameters: the embedding dimension and the subgraph size. The embedding dimension refers to the size of the vector representation for each node, which influences the model’s ability to capture and represent the complex structure of the graph. The subgraph size determines the number of nodes considered within each subgraph, affecting how local structures are utilized during the clustering process.

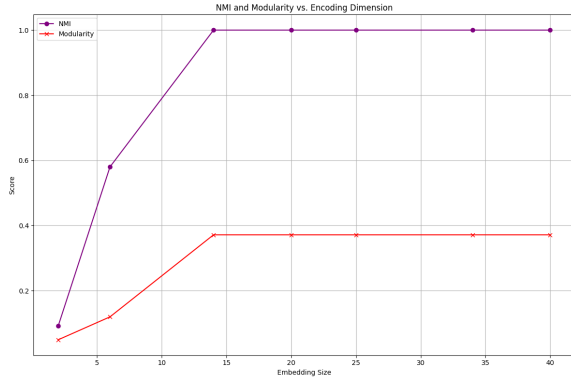


Figure 3: Variation of modularity and nmi with embedding size values for karate dataset

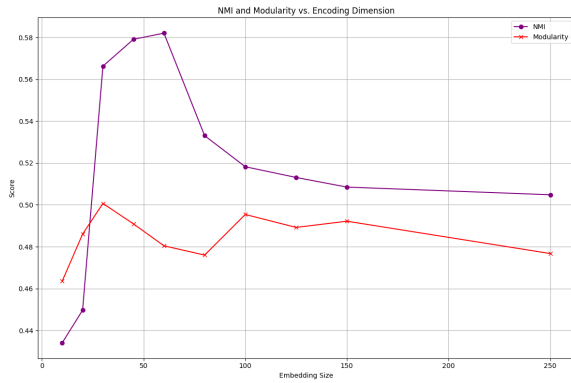


Figure 4: Variation of modularity and nmi with embedding size values for Poolbooks dataset

By analyzing the impact of embedding vector size on community detection, we observe that the performance, measured by Normalized Mutual Information (NMI) and modularity, varies with the structure of the network. Dense graphs tend to have stable and higher NMI and modularity values in high-dimensional, indicating that embeddings are effectively capturing community structures. Sparse graphs, however, show more variability with low-dimensional embeddings, while very high dimensions can introduce noise. Generally, embeddings with dimensions close to half the size of the graph provide a good balance, yielding better performance. This suggests that mid-dimensional embeddings have sufficient representation capacity, making them optimal for capturing community structures in various network types. In LFR datasets with varying mixing parameters, the algorithm demonstrates greater stability when applied to datasets with lower mixing parameters, particularly at higher embedding dimensions.

The analysis of the results in the variation of the number of nodes in the subgraph for the polbooks dataset indicates that the compression algorithm effectively reduces noise, and using a subgraph instead

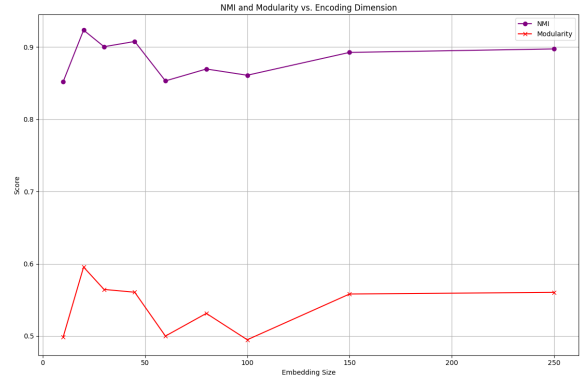


Figure 5: Variation of modularity and nmi with embedding size values for football dataset

of a full graph enhances clustering accuracy and community identification as it is the case for the polbooks dataset. However, it's crucial to retain enough nodes in the subgraph to preserve sufficient information about the graph's community structure. Figure 8 shows the variations of the NMI score for different mixing parameters on the LFR benchmarks and we can see that the bigger the mixing parameter is the bigger the needed number of nodes in the subgraph is to get good performance.

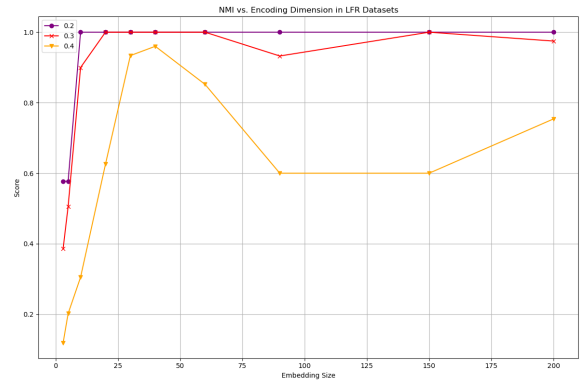


Figure 6: Variation of NMI with embedding size values for LFR dataset with different mixing parameters

5.5 Comparison of Core Components

Figure 9 illustrates the comparison of the effect of each component of our method, including the embedding and the graph compression across networks based on NMI. The figure shows Spectral Clustering (SC), MVE combined with SC (MultiView Embedding + SC), and MVEGC(MultiView Embedding Graph Compression).

- Spectral Clustering (SC): Typically results in lower NMI scores across the datasets due to its limitations in handling complex network structures with-

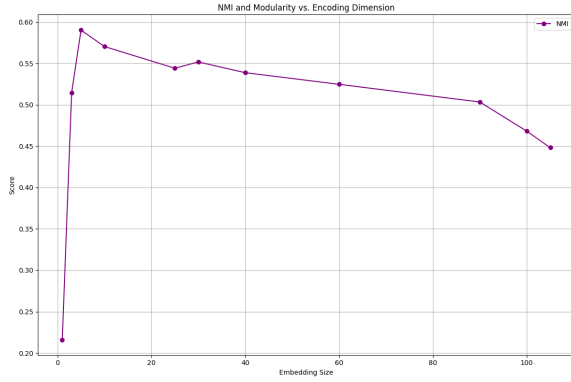


Figure 7: Variation of NMI with subgraph size for the Polbooks dataset.

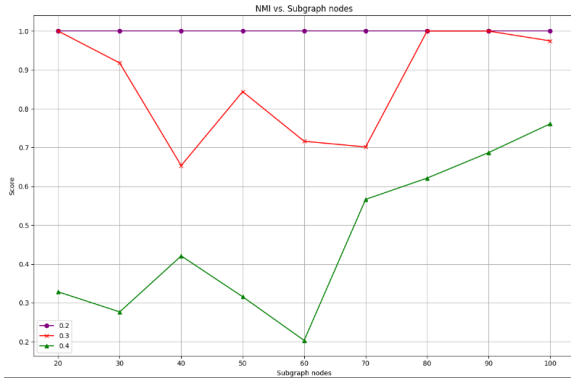


Figure 8: Variation of NMI with subgraph size for LFR benchmarks.

out additional embedding techniques. - MVE + SC: Shows improvement in NMI scores compared to SC alone, indicating that the embedding process helps in capturing the network’s structure more effectively. - MVEGC: Demonstrates the highest NMI scores in most cases, highlighting the effectiveness of integrating embedding and graph compression into a unified approach.

5.6 Comparison with other methods

In this section, we will compare our method with various existing methods, starting with those that utilize well-known embedding techniques (MVEGC, DeepWalk, Node2Vec, and NMF). We will use both real datasets (Figure 10) and synthetic datasets (Figure 11).

Our method demonstrates superior performance in almost all real graph datasets, outperforming other methods significantly. For synthetic datasets, our approach consistently delivers the best results. However, beyond the value of 0.45 for the parameter μ , we observe a slight drop in performance. This can be attributed to the fact that the community structure be-

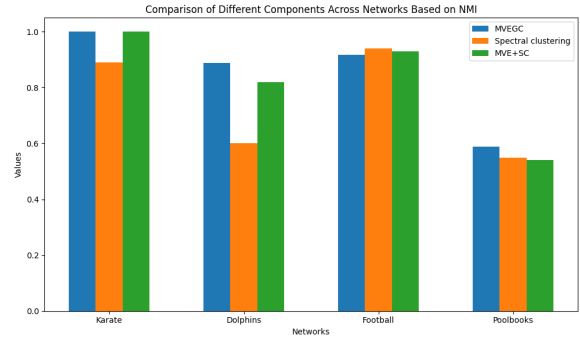


Figure 9: Comparison of NMI performance for Spectral Clustering (SC), MVE + SC, and MVEGC.

comes increasingly difficult to detect above this value, combined with the inherent randomness of these synthetic datasets. Despite this, our method achieves appreciable results compared to other methods

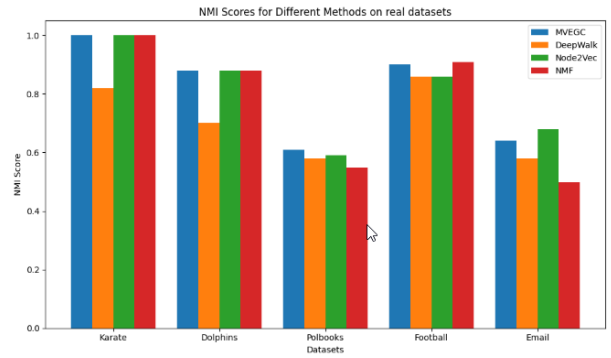


Figure 10: NMI Scores for Different Methods and Datasets. This figure compares the NMI scores of MVE, DeepWalk, Node2Vec, and NMF across various datasets.

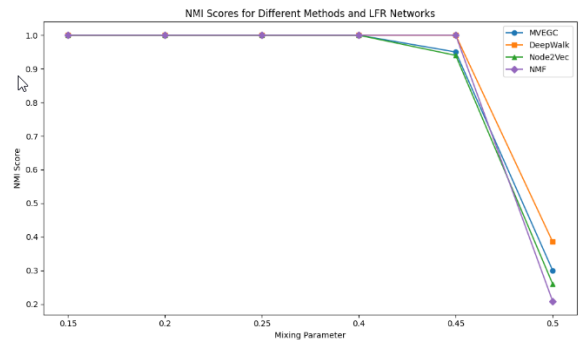


Figure 11: Comparison of NMI Scores for Embedding Techniques on LFR.

Finally, Figure 1213 compares respectively the modularity and NMI performance of MVEGC with other clustering methods: FN (Newman, 2004), SC (Amini et al., 2013), Edmot (Li, Wang, & Lai, 2019), and DSACD (Fei et al., 2020) across real networks.

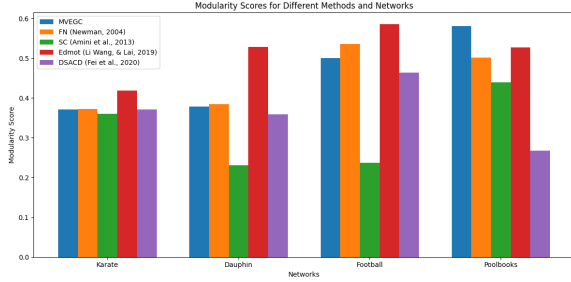


Figure 12: The modularity performance of MVEGC compared to other clustering methods across various networks.

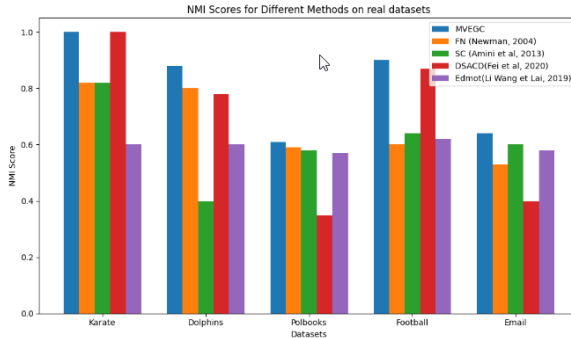


Figure 13: The nmi performance of MVEGC compared to other clustering methods across various networks.

MVEGC demonstrates consistent performance across all networks, as evidenced by the high NMI scores, showcasing the effectiveness of our approach. The core strategy of our method focuses on clustering core nodes first. Since core nodes have the most significant influence on their neighbors, initiating the clustering process from these nodes allows us to achieve satisfactory results with fewer iterations. This approach enables us to disregard unnecessary information right from the initial stages of clustering, contributing to higher accuracy, as reflected in the superior NMI scores compared to other methods.

While our primary objective was not to maximize modularity, the slightly lower modularity values are foreseeable and do not detract from the method’s overall effectiveness. The combination of our strategic clustering and the type of embeddings used enhances the final results by providing our algorithm with a comprehensive understanding of the graph’s information.

This effectiveness is particularly evident in the results obtained from the Poolbooks dataset. Known for its complex structure and misleading nodes, the high NMI scores achieved demonstrate the robustness of our method. Similarly, MVEGC achieved high scores for the Karate and Dolphin datasets, further validating our approach.

On the other hand, results on the LFR dataset are

highly correlated with the number of selected core nodes. As the mixing parameter increases, introducing nodes with higher degrees necessitates the selection of more core nodes. This correlation underscores the adaptability and precision of our method for this latest.

6 CONCLUSIONS

The proposed Multi-View Embeddings Graph Compression (MVEGC) framework effectively integrates multiple similarity matrices, compresses the graph to retain its essential structure, and performs a two-stage clustering process. This approach leverages the rich information provided by different views of the graph and the efficiency of graph compression to achieve high-quality community detection. Experimental results on various benchmark datasets demonstrate the robustness and effectiveness of MVEGC.

Future directions for research include optimizing the framework for scalability to handle even larger networks through parallelization and distributed computing strategies. Additionally, integrating dynamic similarity measures and exploring hierarchical clustering approaches could enhance the framework’s versatility and robustness across different types of real-world networks. The promising performance of MVEGC on diverse datasets suggests its potential applicability to a wide range of complex, large-scale network analysis tasks.

REFERENCES

- Adamic, L. A. and Adar, E. (2003). Friends and neighbors on the web. *Social Networks*, 25:211–230.
- Amini, A. and Levina, E. (2013). On semidefinite relaxations for the block model. *Annals of Statistics*, 41(4):2097–2122.
- Blondel, V. D., Deville, J.-L., Golub, N., and Dooren, P. V. (2008a). Fast unfolding communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10):P10008.
- Blondel, V. D., Guillaume, J.-L., Lambiotte, R., and Lefebvre, E. (2008b). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10):P10008.
- Chen, Y., Wang, L., Qi, D., Ma, T., and Zhang, W. (2020). Community detection based on deepwalk model in large-scale networks. *Security and Communication Networks*, pages 1–13.
- Clauset, A., Newman, M. E. J., and Moore, C. (2004). Finding community structure in very large networks. *Physical Review E*, 70(6):066111.

- Danon, L., Díaz-Guilera, A., Duch, J., and Arenas, A. (2005). Comparing community structure identification. *Journal of Statistical Mechanics: Theory and Experiment*, 2005(09):P09008.
- Fei, M., Wang, S., Zhang, X., Zhou, F., and Zhang, H. (2020). Dsacd: Deep sparse autoencoder based community detection. *Knowledge-Based Systems*, 187:104813.
- Fei, R., Wan, Y., Hu, B., Li, A., and Li, Q. (2023). A novel network core structure extraction algorithm utilized variational autoencoder for community detection. *Expert Systems With Applications*, 222:119775.
- Fortunato, S. (2010). Community detection in graphs. *Physics Reports*, 486(3-5):75–174.
- Gillis, N. and Glineur, F. (2012). Accelerated multiplicative updates and hierarchical als algorithms for non-negative matrix factorization. *Neural Computation*, 24(4):1085–1105.
- Grover, A. and Leskovec, J. (2016a). node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, pages 855–864. ACM.
- Grover, A. and Leskovec, J. (2016b). node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 855–864.
- Hancock, J. M. (2004). Jaccard distance (jaccard index, jaccard similarity coefficient). In *Dictionary of Bioinformatics and Computational Biology*, page n/a. Wiley.
- He, H., Zhao, Z., Luo, W., and Zhang, J. (2021). Community detection in aviation network based on k-means and complex network. *Not Published, Not Applicable*(Not Applicable):Not Applicable. Received: 26 January 2021; Accepted: 16 March 2021.
- Idocin, J. F., Córdón, , Minárová, M., Betanzos, A. A., and Bustince, H. (2022). Combinations of affinity functions for different community detection algorithms in social networks. In *Proceedings Of The . . . Annual Hawaii International Conference On System Sciences*.
- Lancichinetti, A., Fortunato, S., and Radicchi, F. (2008). Benchmark graphs for testing community detection algorithms. *arXiv preprint arXiv:0805.4770v4*. Submitted on 30 May 2008 (v1), last revised 30 Oct 2008 (this version, v4).
- Leng, M., Yao, Y., Cheng, J., Lv, W., and Chen, X. (2013). Active semi-supervised community detection algorithm with label propagation. *JSchool of Information Science and Engineering, Lanzhou University Lanzhou, 730000, China*.
- Leng, M., Yao, Y., Cheng, J., Lv, W., and Chen, X. (2024). Active semi-supervised community detection algorithm with label propagation. *School of Information Science and Engineering, Lanzhou University*.
- Li, A., Huang, Z., Wang, Y., and Lai, Y.-C. (2019). Edmot: An edge enhancement approach for motif-aware community detection. *IEEE Transactions on Cybernetics*, 49(12):4451–4464.
- Li, Z., Zhang, S., Wang, R.-S., Zhang, X.-S., and Chen, L. (2008). Quantitative function for community detection. *Physical Review E*, 77(3):036109.
- Louaf, W. and Titouna, F. (2023). PCMeans: Community detection using local pagerank, clustering, and K-means. *Social Network Analysis and Mining*, 13:103.
- Manning, C. D., Raghavan, P., and Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
- Mazouzi, D., Rhouma, D., and Ben Romdhane, L. (2019). Community detection in large-scale social networks: State-of-the-art and future directions. *Social Network Analysis and Mining*, 9:1–32.
- Newman, M. E. J. and Girvan, M. (2004). Finding and evaluating community structure in networks. *Physical Review E*, 69(2):026113.
- Perozzi, B., Al-Rfou, R., and Skiena, S. (2014). Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 701–710.
- Raghavan, U. N., Albert, R., and Kumara, S. (2007). Near linear time algorithm to detect community structures in large-scale networks. *Physical Review E*, 76(3):036106.
- Rozemberczki, B., Davies, R., Sarkar, R., and Sutton, C. (2019). GEMSEC: Graph embedding with self clustering. In *International Conference on Advances in Social Networks Analysis and Mining*, Vancouver, British Columbia, Canada.
- Shigehalli, V. S. and Shettar, V. M. (2011). Spectral technique using normalized adjacency matrices for graph matching. *International Journal of Computational Science and Mathematics*.
- Su, Y. et al. (2017). A seed-expanding method based on random walks for community detection in networks with ambiguous community structures. *Scientific Reports*, 7:41830.
- Subramani, K., Velkov, A., Ntoutsis, I., Kröger, P., and Kriegel, H.-P. (2011). Density-based community detection in social networks. *Institute for Informatics, Ludwig-Maximilians-Universität München, Munich, Germany*.
- Sui, S.-K., Li, J.-P., Zhang, J.-G., and Sui, S.-J. (2016). The community detection based on svm algorithm. In *Proceedings of the IEEE International Conference on Computer, Wireless and Mobile Technologies in Education (ICCWAMTIP)*.
- von Luxburg, U. (2007). A tutorial on spectral clustering. *Statistics and Computing*, 17(4):395–416.
- Wang, X., Shen, G., Zhang, Z., Lai, S., Zhu, S., Chen, Y., and Quan, D. (2024). Self-supervised graph autoencoder with redundancy reduction for community detection. *College of Information Engineering, China Jiliang University*.
- Xie, Y., Wang, X., Jiang, D., and Xu, R. (2019). High-performance community detection in social networks using a deep transitive autoencoder. *Information Sciences*, 493:75–90.

- Xu, R., Che, Y., Wang, X., Hu, J., and Xie, Y. (2020). Stacked autoencoder-based community detection method via an ensemble clustering framework. *Information Sciences*, 526:151–165.
- Zhang, J., Fei, J., Song, X., and Feng, J. (2021). An improved louvain algorithm for community detection. *Complexity*, 2021:1–15. Received 12 September 2021; Revised 17 October 2021; Accepted 9 November 2021; Published 23 November 2021.
- Zhao, X., Li, Y., and Qu, Z. (2018). A density-based clustering model for community detection in complex networks. *AIP Conference Proceedings*.